Smart University
Resource Allocation System

Iteration 42
Student-Professor Appointment Scheduling

Najib Attar | UC-STU-01 | October 2026

1. Iteration Goal

Deliver the complete approved appointment use case while preserving existing weekly resource allocation. Students and professors work with real dates; C++ determines feasibility and recommendations, Flask controls authenticated workflows, and PostgreSQL stores appointments and safe reservations.

2. Changes from Iteration 41

Iteration 41 baseline	Iteration 42 addition
PostgreSQL authority; 18 application tables	10 additive identity/calendar/appointment tables; 28 total
Weekly room-allocation JSON v1	Separate appointment JSON v1 with UTC dated intervals
Technical dashboard and weekly schedules	Authenticated participant portal, office hours and dated calendar
Structured allocation results and safe publication	Revision-specific approval, per-person overlap protection and cancellation

3. Scenario

A student selects a configured faculty and one or more professors, enters a purpose and duration, and optionally invites other students. The system presents feasible times, ranked deterministically. Pending submission reserves nothing. After every required professor approves the same revision, availability is rechecked and mandatory commitments are confirmed together. Optional students may decline without cancelling the meeting.

One student/one professor, one student/multiple professors, multiple students/one professor and multiple students/multiple professors are supported. Time changes reset approvals; the organizer or a selected professor may cancel and release confirmed time.

4. Modules and Their Tasks

Component	Responsibility
AccountService + security module	Trusted account provisioning, scrypt password verification, revocable signed sessions, CSRF and role enforcement
Appointment blueprint	Student/professor forms, request/status pages, invitations, office hours and own-calendar views
AppointmentService	Participant authorization, lifecycle, revisions, approvals, invitations and short database transactions
calendar_service	Explicit timezone conversion and expansion of bounded patterns, exceptions and legacy period bindings; no scheduling decisions
EngineGateway	Verified executable, appointment JSON transport, timeout/exit and structured response validation
AppointmentJsonProtocol	Separate versioned stdin/stdout operation; typed input validation and structured results
AppointmentSchedulingService	Mandatory feasibility, half-open interval conflicts and deterministic feasible-time ranking
PostgreSQL + Alembic	Persistent calendar/approval/history data, deferred aggregate integrity and GiST overlap protection

5. Updated Architecture

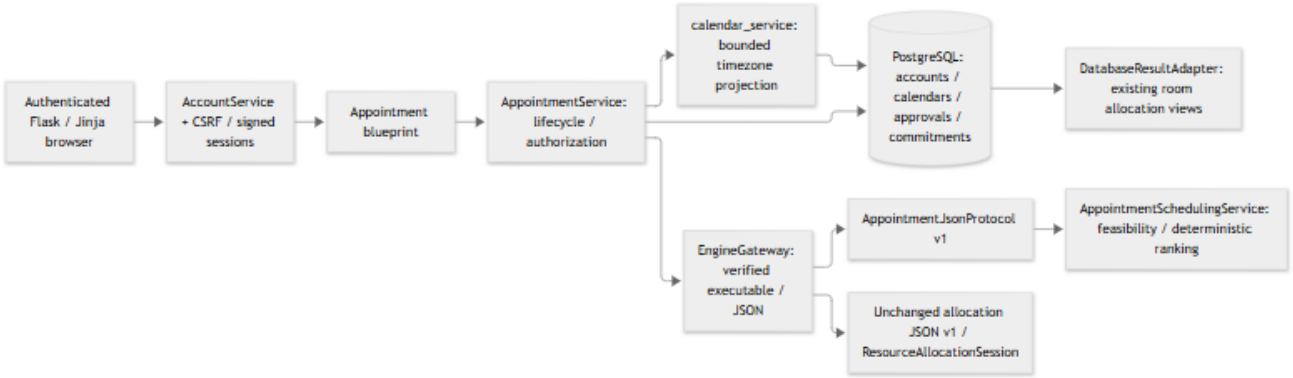


Figure 1. Appointment boundary alongside the unchanged allocation engine.

The gateway remains an adapter/indirection boundary. Focused services are pure fabrications supporting high cohesion; C++ is the information expert for feasibility. Existing allocation Strategy, factories and facades are reused unchanged. No new pattern framework or direct SQL dependency was introduced in C++.

6. Database and Calendar Design

New table	Important purpose / relationship
login_account	Optional account per university user; unique username, hash, session version and explicit administration grant
academic_period	Known inclusive start/end dates and recorded timezone
legacy_calendar_binding	Exactly one busy-slot or resource-request association; optional planning week and exceptions
professor_availability	Professor weekday/minute pattern with explicit validity and excluded dates
calendar_block	Date-specific personal/professor unavailability
appointment	Organizer, purpose, duration, aware interval, lifecycle and revision
appointment_professor	Required professor decision on the current revision
appointment_invitation	Optional student invitation; unanswered or declined
appointment_commitment	Exact confirmed organizer/professor reservations; per-person exclusion constraint
appointment_event	Participant-visible actor/revision history

Alembic revisions 0042a and 0042b are additive. Original allocation tables remain intact. Deferred checks require valid participant roles, current professor revisions and unanimous approvals before confirmation. Confirmed commitment rows must match the appointment interval and mandatory-person set; other states cannot reserve time.

Europe/Istanbul is explicitly configurable. Dated timestamps are timezone-aware and intervals half-open [start, end), permitting adjacent meetings. Ambiguous or nonexistent wall times are rejected. Existing TimeSlot weekday/minute behavior is unchanged.

Undated records are never assigned guessed dates. An administrator associates them with an explicit academic period, bounded weekly recurrence or a specified planning week, and exceptions. Unknown mandatory applicability returns UNRESOLVED_CALENDAR; unknown optional invitees are not counted known-free. Attendee counts are not an enrollment roster.

7. Updated Workflow and Prototype

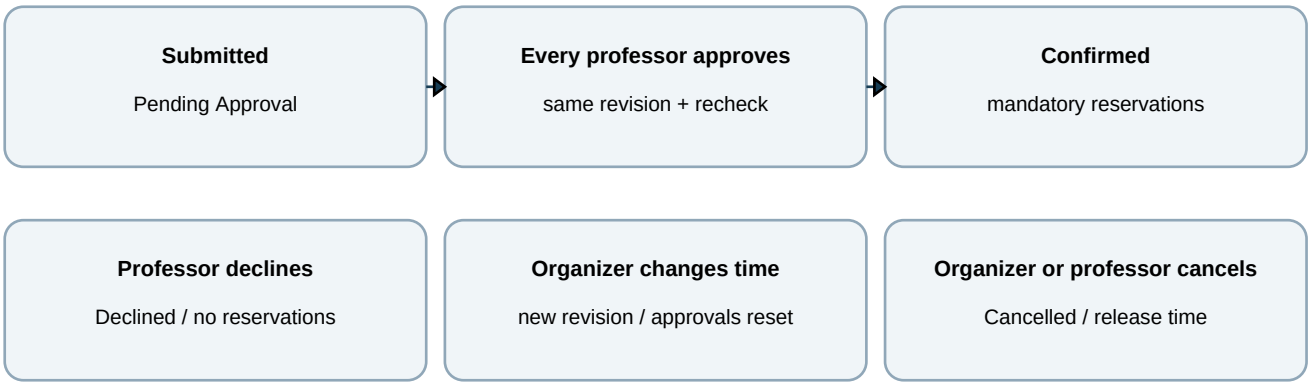


Figure 2. Lifecycle paths; only Confirmed creates mandatory commitments.

C++ first checks publication coverage, blocked periods, known university/appointment commitments, organizer and every required professor, and duration. Feasible candidates are ranked by known-free invited students, preferred-time distance and chronology. A 15-minute recommendation grid never relaxes feasibility; valid manual proposals need not match that grid.

Engine evaluation occurs outside database transactions. Confirmation compares calendar and appointment revisions under a short state-row lock, then writes decisions, state, history and reservations atomically. PostgreSQL rejects overlapping mandatory-person ranges, including competing worker processes. Room-result publication cannot introduce mapped commitments conflicting with confirmed appointments.

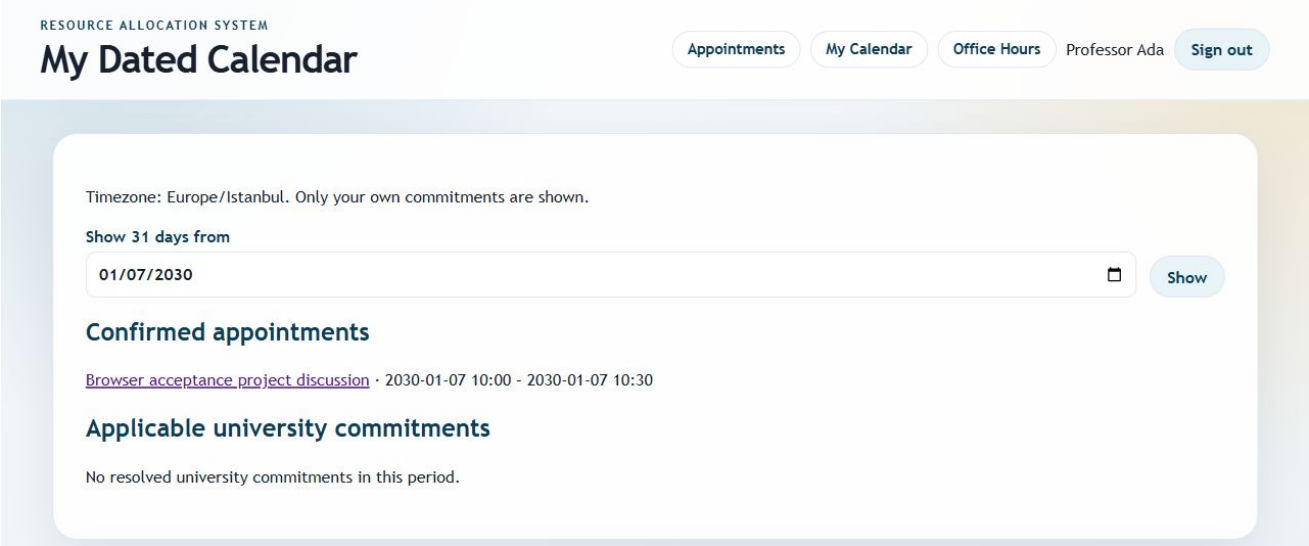


Figure 3. Isolated fictional browser acceptance: confirmed appointment in the professor's dated calendar.

Student and professor navigation remains Flask/Jinja/vanilla JavaScript. Private appointment details are participant-scoped. The old technical dashboard requires a separately trusted administration grant; both its existing forms and new appointment forms are protected against CSRF.

8. Verification

Check	Executed result / evidence
Fresh C++ build + complete suite	148 passed, 0 failed: 140 preexisting and 8 new appointment tests
Complete Python suite	191 total: 190 passed, 0 failed, 1 Windows skip for Linux execute-permission semantics
Real PostgreSQL	Fresh additive migration, head 0042b, no Alembic drift; integrity, rollback, concurrent writers and recovery exercised
Engine compatibility	All five allocation strategies retain representative CSV/JSON/database parity; four legacy request types remain supported
Appointment acceptance	All four participant configurations, mandatory/optional conflicts, revision reset, rejection, invitation decline, cancellation and two-process confirmation
Protocol / security	Malformed input/output, version/correlation, timeout/exit; hashed login, revocation, CSRF, ownership and schedule privacy
Browser + responsive review	Student sign-in/selection/recommendation/submission, professor approval/publication, confirmed calendar; narrow viewport checked
Legacy and file safety	Two isolated CLI smoke scenarios; original generated CSV hashes unchanged; no automatic legacy date backfill
Static/document checks	Python compile, JavaScript syntax, actionlint, model/schema catalog checks and diagram generation

Commands: powershell -ExecutionPolicy Bypass -File tests\run_tests.ps1; python -m unittest discover -s tests -p 'test_*.py' -v; python -m alembic check; python scripts/generate_database_diagrams.py --check. PostgreSQL tests own disposable isolated schemas; browser acceptance uses a separate fictional development dataset. Remote PR checks are reviewed separately; the report records local verification.

9. Remaining Scope and Summary

Trusted account provisioning is required; there is no public self-registration, password-recovery service or email delivery. Activity and status views provide in-application visibility. Academic periods and faculty hierarchy must be configured from known information. Removing office hours conservatively requires resolving the professor's future confirmed appointments first.

Search is bounded to 32 days and 20 returned recommendations by default, publication patterns to two years and duration to one day. Existing anonymous attendee counts cannot identify individual student class commitments. These explicit limitations do not fabricate calendar facts or defer the approved approval/invitation/cancellation behavior.

Iteration 42 adds a complete dated appointment workflow without rewriting allocation algorithms, introducing a C++ database dependency or replacing the frontend stack. PostgreSQL remains the sole operational authority; weekly and dated schedules coexist through explicit calendar configuration. The branch is prepared for review, not merged or deployed by this report.

Iteration 42 - Final Portal Refinements

Final verification supplement | October 14, 2026

The original report describes the appointment implementation baseline. This supplement records the final student portal and demonstration improvements included in the same iteration. Earlier verification totals are superseded by the final totals below.

Student Portal and Schedule

Students now land on Home after signing in. One PostgreSQL snapshot supplies the next confirmed meeting, pending requests, optional invitations, organizer-owned bookings and a private timetable. All university roles can view their own current-month schedule with Monday-Friday columns, hourly rows, week navigation, precise event intervals and a separate weekend list.

Appointment entry uses separate searchable professor/student lists and one date. Feasible time buttons are generated by the existing C++ scheduler. Changing participants, date or duration clears stale selections. Manual same-date proposals receive the same submission and confirmation checks. Known fictional-name suffixes are hidden only in presentation; stored identities remain unchanged.

Component	Responsibility
StudentDashboardService	Private summary counts and cards in one read-only snapshot
PersonalScheduleService	Shared dated timetable projection; no duplicate booking data
presentation.person_name	Display-only formatting for known fictional demo names
run_demo / setup_local_demo	Safe local setup, fictional users and real-engine allocation

Demonstration and Verification

A one-command Windows launcher prepares a separate password-protected, loopback-only PostgreSQL demo with 80 fictional students, 12 professors, 20 rooms and 120 resource requests. The engine produces allocations. Explicit January 2030 dates are fixture data, not guessed dates for existing records. Repeated setup preserves accounts and data; generated credentials stay private.

Final local results: freshly compiled C++: 148 passed, 0 failed; Python: 224 executed, 223 passed, 0 failed, 1 Windows skip for Linux executable-permission semantics. Real PostgreSQL tests cover migration/drift, recovery, five-strategy parity, rollback, process concurrency, authentication and appointment workflows. Both isolated CLI smokes passed. Syntax, templates, schema diagrams and original CSV hashes were verified.

Browser acceptance verified login, populated and empty Home, precise timetable entries, participant search, stale-slot invalidation, one-date submission, two-professor confirmation and a narrow responsive viewport. PostgreSQL remains the sole operational authority; all 28 tables and Alembic head 0042b are unchanged by these refinements.

Remaining boundaries: schedules contain only known participant identities with explicit calendar applicability. Anonymous attendee counts are not course-enrollment rosters. Trusted account provisioning and local-demo tooling do not constitute production deployment. Existing allocation algorithms and both JSON operations remain unchanged.